

面试押题 01: FA3 确定性 SWA 反向优化

对应九坤实习: FA3 确定性 SWA backward 调度。核对日期: 2026-09-29。主线采用已经验证并交付的 dev/m2 候选; 9 月 20 日泛化实验单列, 不能混成同一组收益。

0. 开源仓库技术介绍与应用

FlashAttention 是精确注意力的高效 GPU 实现: 通过分块和融合计算减少 HBM 中间张量读写, 不把完整注意力矩阵写回显存。FA3 面向 Hopper, 利用异步数据搬运和矩阵计算重叠提高利用率。

典型应用: Transformer 训练的 attention forward/backward, 以及长序列推理。它是算子库, 不负责整网训练调度。我的工作针对内部带 sink、滑窗和确定性约束的 backward 调度, 不能把上游全部功能或收益归为个人贡献。

1. 先用一分钟讲清楚 (STAR)

背景 S: 在 IQuest-Q1 (320B-A15B) 研发阶段, 长序列、带 sink 的滑动窗口注意力开启确定性反向后明显变慢, 线上观测单 microbatch forward 约 9 ms、backward 约 58 ms; 这只是排障入口, 不能直接用两个数计算优化收益。

任务 T: 我负责从训练 trace 找出具体瓶颈, 在不取消确定性、不删掉 sink、不改变公开 API 的前提下优化 FA3, 并完成正确性、性能和交付验证。

行动 A: 我先固定源码与输入, 将 dQ、dK、dV 的顺序化拆成三个编译期变量做全因子对照, 再用 Nsys 和 NCU 确认主要问题是 dQ 跨 CTA semaphore 等待; 最终在原 fused backward 内把绝对 ticket 改为有效 contributor 的反向相对编号, 并让 scheduler 的执行顺序与 ticket 对齐, 短序列与不支持配置保留 native 回退。

结果 R: 带 sink 的代表 packed shape, 完整 FA backward 从 6.755 ms 降至 1.699 ms, 约 3.98 $\times$; 2K-12K 范围为 1.59-5.37 $\times$ 。完成 3,601 项仓库回归、目标路径 1,000 次逐位确定性检查及 wheel 交付。这里是 FA backward 收益, 不是整网训练加速 3.98 $\times$ 。

2. 系统背景、职责与改动边界

FA3 承担注意力核心计算; SWA 只允许每个 Query 访问局部 Key 窗口; varlen 将不同长度样本打包; GQA 让多个 Q heads 共享 KV heads。目标配置是 H200/SM90、BF16、head dim 128、Q16/KV2、左窗口 512、右窗口 0、deterministic-on、sink-on。

同一 dQ tile 会收到多个 K-block CTA 的梯度贡献, 浮点加法不满足结合律。原实现用 semaphore 固定 reduce-add 顺序, 保证运行间确定性。我的工作不是发明 FlashAttention, 也不是删掉同步, 而是重新组织已有 fused kernel 的依赖链和调度, 并建立可复现的验收证据。

生产候选在 dev/m2 上只改两个文件: `hopper/flash_bwd_launch_template.h` 与 `hopper/mainloop_bwd_sm90_tma_gmma_ws.hpp`; 相对基线为 62 行增加、10 行删除, 保留原 shared-memory pipeline、dK/dV、sink 与 Python 接口。

3. 根因: 有 warp 驻留, 但没有足够可发射 warp

原路径对每个 dQ tile 使用绝对 `n_block` 作为 ticket。SWA 下真正贡献的 K blocks 只占一小段, 但无贡献 block 仍要推进空 ticket; 同时, 后继 CTA 可能先驻留, 等待尚未被有效调度的前驱。

其关键顺序是: `wait_eq(ticket) → TMA reduce-add → 等待 store 完成 → arrive_inc`。不能简单删除 `wait`, 否则固定累加顺序就丢失; 也不能在数据写完前发布 `arrive`, 否则后继可能观察到不完整结果。

诊断证据	观测	能支持的结论
三因素对照	保留 host/workspace、关闭顺序化约 1.138 ms; 仅启用 dQ 约 6.411 ms	性能台阶主要由 dQ 引入
Shapley 归因	代表 shape 中 dQ 占顺序化增量 99.67%	是该实验的归因占比, 不是全训练耗时占比
NCU eligible warps	0.412 → 0.052	已驻留工作大量不可发射
NCU barrier stall	1.50 → 41.53 cycles/issued instruction	等待链成为主要瓶颈
DRAM 带宽	975 → 132 GB/s	慢路径并非把带宽跑满

最重要的推理链: TMA `reduce-add` 在对照组和慢路径都存在, 变化是它前后的顺序控制; 慢路径带宽和 TMA 活跃度反而降低, 源码相关性又命中 `Barrier::wait_eq`, 因此优先改 `semaphore` 与 `scheduler`, 而非先换 GEMM tile。

4. 我如何实现, 以及为什么放弃其他方案

- 对齐基线。**固定 FA、CUTLASS commit 和真实 packed lengths, 确认 sink、deterministic 等开关, 不拿不同分支的时间直接相减。
- 做诊断变量。**分别控制 dQ/dK/dV 顺序化, 保留其余 host 与 workspace 开销, 隔离影响。
- 验证方向。**query-major Triton 原型让一个 CTA 独占 dQ tile, 证明消除跨 CTA 依赖有收益; 但它额外启动 kernel, 而且原 fused 路径仍重复计算 dQ, 因此没有直接作为交付方案。
- 做单主 kernel 消融。**比较 Relative Ticket、Swizzled Absolute、Forward-Aligned 和 Reverse-Aligned。只缩短 ticket 或只换调度都不等于最终组合; Forward-Aligned 保留旧 dQ 累加顺序, 但长序列收益不足。
- 选择 Reverse-Aligned。**对某个 Q tile 的有效贡献区间 `[n_min, n_max)`, 反向编号为 `ticket = n_max - 1 - n_block`; 让 reverse scheduler 按同序推进, 并交错不同 batch/head 的独立链。没有贡献的 CTA 不再推进空 tail。
- 保护适用范围。**9 月 10 日候选只在已验证的架构、dtype、head/window 和长度条件启用; 判断只读已有 host metadata, 不读取 device `cu_seqlens`, 没有 D2H 同步、额外 workspace 或额外 kernel。
- 做分层验收并打包。**FP32 reference、自身确定性、通用回归、逐阶段 profile、最终 wheel smoke 分开记录。

一个关键取舍: Reverse-Aligned 改变了 dQ 的固定累加顺序, 所以它对自身逐位确定, 却不保证与旧 native 的 dQ 逐位一致; 这是确定性与旧实现逐位兼容两个不同要求, 必须主动说明。

5. 实验矩阵与指标口径

5.1 简历主指标: dev/m2 + sink

环境为北京 H200、CUDA 12.8、PyTorch 2.10.0+cu128; 每点 10 次预热、50 次 CUDA Event 测量、5 个独立进程。基线 `816ca61f`, 候选 `8612871e`。

对象	Baseline	Candidate	指标
packed <code>[8192, 8192, 8192, 4096, 4096]</code> 完整 FA backward	6.7553 ms	1.6989 ms	3.98×; 延迟降低 74.85%

对象	Baseline	Candidate	指标
相同输入的 FA main kernel	6.0715 ms	1.0246 ms	5.93×, 不能替换完整 backward 指标
2K / 4K / 8K / 12K	分点独立比较	分点独立比较	1.59× / 2.30× / 4.00× / 5.37×
512 / 1024	native 回退	native 回退	1.004× / 0.995×, 基本持平

Nsys 中 kernel 类别、次数一致, sink dQ、dSink reduce、pre/post 基本不变, 收益集中在主核。正确性包括: 专用矩阵 12/12、仓库回归 3,601 passed、FP32 reference 阈值 3e-3 内、目标 reverse 固定输入重复 1,000 次, dQ/dK/dV/dSink 各仅一个 hash 且全部 finite。

5.2 线上证据不是严格端到端 A/B

线上 84 次 FA backward 验证了长序列命中 reverse、短/高度 ragged 输入回退 native。两份 trace 的 packing 与 microbatch 数不同, 而且仅两个 ProfilerStep, 因此 FA main 聚合降低 43.8%、归一化 ProfilerStep 降低 4.7%只作趋势, 不用于宣称严格整网加速。

5.3 9月20日更新: 泛化与免预清零

新实验把 head guard 从 Q16/KV2 扩为合法 GQA、窗口扩为左侧局部窗口; 保留 BF16/D128/SM90/varlen/deterministic/softcap=0 等边界。同时在 Megatron 删除公共 BF16 dq/dk/dv 的三次外部预清零, 保留内部 accumulator、semaphore、work counter 的必要初始化。

四组 Q18/KV3 组合实验记录 8.39%-76.28% 延迟改善, 但统计是三个进程各自 P10 的中位数, 用来减轻常驻负载干扰; 不是独占 GPU 普通 median, 也不是完整训练 step。NaN poison 与 1,000 次 hash 检查通过, 不能据此跳过正式生产 A/B。

sink dQ 融合进 FP32 postprocess 的原型虽然精度通过, 却让四组性能回退 8.7%-25.6%, 已拒绝; “做了融合”不等于“交付了加速”。

6. 高频问答 (10 题)

Q1: 为什么 deterministic backward 比非确定性慢?

因为多个 CTA 对同一 dQ tile 的浮点贡献需要固定顺序; 本例用 semaphore 串行化 reduce-add。当 scheduler 与依赖顺序不匹配, 已驻留后继等待前驱, SM 看起来有活跃 warp, 却缺少 eligible warp。确定性要求本身仍保留, 优化的是无效等待和独立链调度。

Q2: 为什么只改相对 ticket 不够?

相对编号可删掉无贡献 block 的空推进, 但不能保证前驱先获得执行机会。诊断中 Relative Ticket 的改善远小于最终方案; 最终必须把 contributor 顺序与 reverse scheduler 对齐, 并分散独立 batch/head 链。

Q3: 会不会死锁? 如何保证没有跳过贡献?

正确性条件是每个有效 contributor 对其 Q tile 恰好贡献一次、ticket 连续且唯一、发布 arrive 前完成对应写入; scheduler 要能持续推进前驱。实现保留原 barrier 内存顺序, 只改有效 contributor 编号与调度。用短尾块、ragged、不同长度、长期重复及超时检查验证, 但有限测试不等于形式化证明所有 shape 无死锁, 所以保留 guard。

Q4: 为什么不取消 deterministic 或直接用 atomicAdd?

那会改变业务要求。这里需要重复运行输出稳定; 仅“误差小”不足以替代确定性。我的方案仍规定每个 dQ tile 的贡献顺序, 只允许不同 tile/独立链之间并行。

Q5: bitwise deterministic 与等于旧结果有何区别？

前者是同一实现、固定输入多次执行逐位一致；后者是新旧算法相同累加次序或恰好相同结果。reverse 改变顺序，所以 dQ 可以与旧版略有差异，同时自身每次完全一致；还要用 FP32 reference 约束误差，而非只比 hash。

Q6: 为什么需要 max length 和 average length 两个 guard？

初版发现“极少长样本 + 大量短样本”可能仅 max 很大，整体却不适合 reverse 的成本结构。旧交付要求 $\max \geq 2048$ 且 $\text{total_tokens}/\text{batch} \geq 1024$ ；后来泛化实验对更多短 ragged shape 证实收益，才在新候选中放开长度门槛。不能把后来的 guard 说成初版就支持。

Q7: 为什么不用 device cu_seqlens 做更准确的成本模型？

在 host 决策读取它可能引入 D2H 和同步，也会破坏热路径及 Graph 友好性；因此先利用已有 max length、total tokens、batch 等 host 参数。更细模型需要评估其额外成本，而不是认为更多 metadata 必然更优。

Q8: 如何证明不是关掉 sink 获得收益？

固定 sink-on，核对前后 Nsys 的 kernel 类别和次数；sink dQ 与 dSink reduce 的耗时基本不变，main kernel 从 6.0715 ms 降到 1.0246 ms，完整 backward 从 6.7553 ms 降到 1.6989 ms。这能把主要变化归到调度改动。

Q9: 为什么 3.98× 没变成整网 3.98×？

FA backward 只是训练一步中的一部分，还存在 MoE、通信、其他注意力和固定成本；优化段占比及 overlap 决定整网收益。现有线上 trace 不同 shape，不足以给严格整网加速；我会报告固定输入算子结果，并要求同数据、同 packing 的长稳态 A/B。

Q10: 你最有价值的失败实验是什么？

query-major 原型证实依赖链方向但引入额外 kernel/重复 dQ，所以回到 fused 实现；后续 sink postprocess 融合数值更好却性能更差，说明额外 Q/sink load、寄存器与同步可能超过少一次写回的收益。我用 profiling 和完整边界计时选择方案，而不以“融合数量”判断成功。

7. 追问备忘与不能夸大的边界

- 若问上线：回答“交付 wheel、线上 trace 看到目标 scheduler 与回退生效”；不要仅靠 kernel 名声称已精确核验所有线上 wheel SHA 或全网稳态收益。
- 若问覆盖范围：3,601 是仓库测试总数，1,000 是目标固定输入重复次数；二者不是 3,601 个不同生产 shape 都重复 1,000 次。
- 若问最新泛化：只称新候选/组合实验，保留低干扰 P10 口径，不冒用初版 3.98× 的基线。
- 若问 CUDA 13.1 wheel：早期重打包只有编译/导入检查，当时本机 driver 不匹配；不能把 CUDA 12.8 GPU 验证借给另一 runtime。
- 若问整体原理：先画出“多个 K CTA → 同一个 dQ tile”的依赖，再解释 ticket 与 scheduler 对齐，避免只背术语。

8. 开发证据与来源

以下均为只读核对的内部开发材料；路径中的“北京/上海”指存放位置，不代表每项实验都在该 CPU 节点执行。

1. 北京，总结与主指标：[/volume/pt-train/users/zhaoye/doc/M3 问题优化/fa3_backward_analysis/FA3_deterministic_SWA_backward线上问题与解决报告_2026-09-14.md](/volume/pt-train/users/zhaoye/doc/M3%20问题优化/fa3_backward_analysis/FA3_deterministic_SWA_backward线上问题与解决报告_2026-09-14.md)。

- 本次已重新连通上海 CPU 并检索其开发文档；上述 FA3 训练调度的完整一手证据在北京目录，不将上海 serving FA3 消融报告替代为本项证据。